

Programmazione 1

Corso c

04/12/2023

> Ricorsione e correttezza (3)

Alessandro Mazzei

alessandro.mazzei@unito.it

Dipartimento di Informatica

Università degli Studi di Torino



UNIVERSITÀ
DI TORINO

Outline

- Induzione per la correttezza della ricorsione
- Ricerca dicotomica ricorsiva e correttezza
- Merge Sort e correttezza

Come si progetta un metodo ricorsivo

- Vogliamo calcolare una funzione matematica f definita su numeri naturali usando un metodo F ricorsivo.
- Il metodo F deve definire almeno due casi:
 - Caso base della ricorsione: $n == 0$, in questo caso si restituisce il valore di $f(0)$.
 - Passo ricorsivo: $n > 0$, in questo caso la tecnica permette l'uso dell'invocazione $F(n-1)$ per calcolare il valore di $F(n)$.

Si può assumere che questa invocazione restituisca correttamente il valore $f(n-1)$ quindi, se abbiamo scritto con attenzione il codice per il passo ricorsivo, **mediante induzione si dimostra allora che $F(n)$ calcola correttamente il valore di $f(n)$.**

Ricorsione e Induzione

Ricorsione: F

- Base della ricorsione: se $n=0$ si restituisce il valore $f(0)$
- Passo ricorsivo: se $n>0$, si restituisce un valore rappresentato da una espressione che, solitamente, conterrà la sottoespressione $F(n-1)$

L'espressione per il valore di $F(n)$ è suggerita da una definizione ricorsiva per la funzione f .

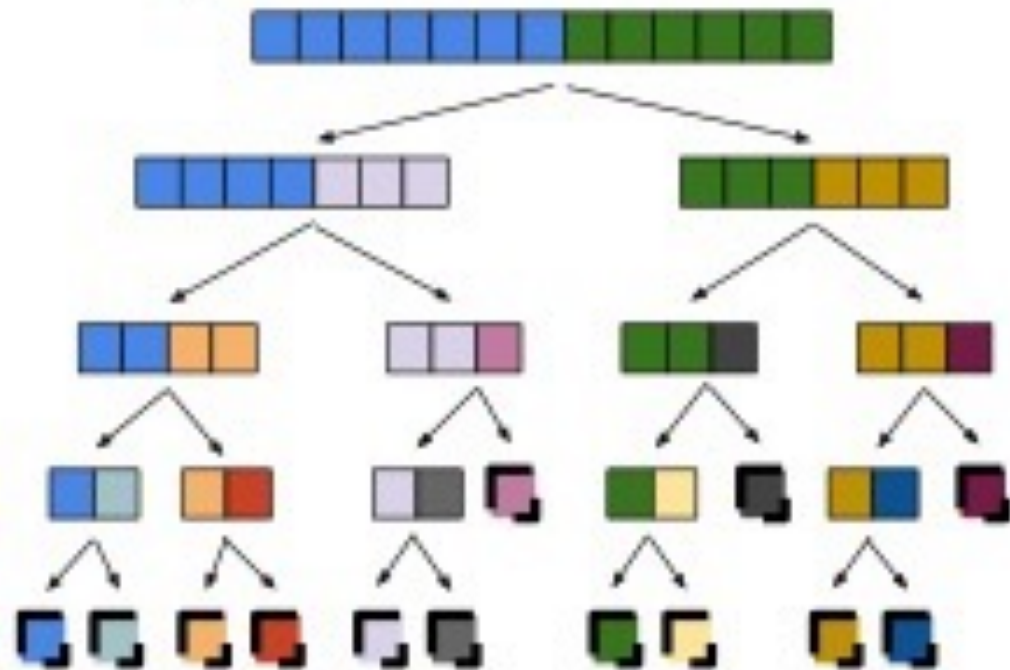
Induzione:

- $F(0)=f(0)$
- Passo induttivo: assumendo che $F(n-1)$ calcoli correttamente $f(n-1)$ (ipotesi induttiva), si dimostra che $F(n)$ calcola correttamente $f(n)$.

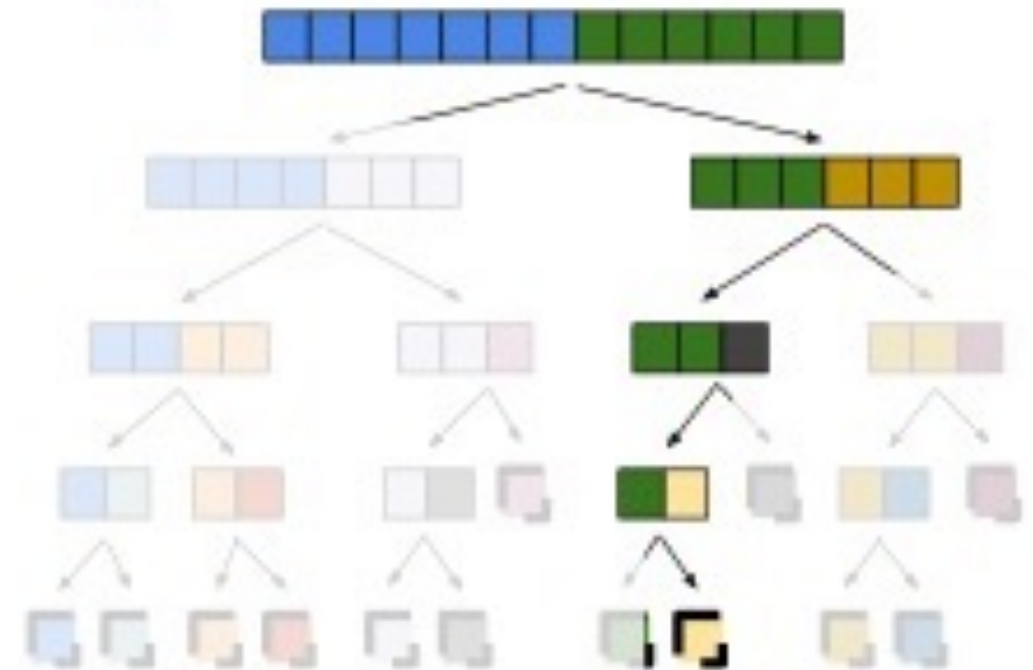
La dimostrazione del passo induttivo sfrutta la definizione ricorsiva della funzione F

Ricerca dicotomica

Esempio ricorsione dicotomica:



Esempio ricerca dicotomica:

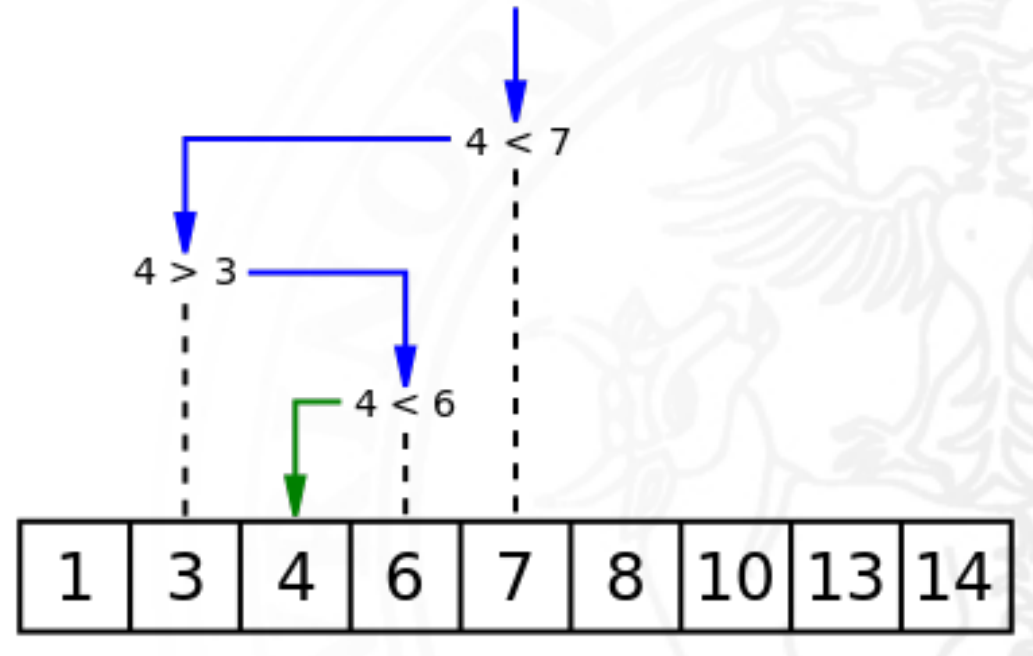


Ricerca dicotomica

- L'algoritmo di ricerca binaria cerca un elemento in una sequenza ordinata in maniera crescente (o decrescente)
- L'algoritmo è simile al metodo usato trovare una parola sul dizionario: l'idea è quella di iniziare la ricerca dall'elemento centrale.
- Si confronta questo elemento con quello cercato:
 - se corrisponde, la ricerca termina;
 - se è superiore, la ricerca viene ripetuta sugli elementi precedenti (ovvero sulla prima metà del dizionario);
 - se invece è inferiore, la ricerca viene ripetuta sugli elementi successivi (ovvero sulla seconda metà del dizionario)
- La ricerca binaria non usa mai più di $\log_2 N$ confronti.

Ricerca dicotomica

- Ricerca dell'elemento 4:



Ricerca dicotomica

```
int ricercaBinariaDic(const int len, const int a[],const int valore,const int
                        left,const int right) {
    if(right==left+1) {
        if(a[left]==valore) {
            return left;
        }
        else {
            return -1;
        }
    }
    else {
        int middle = (left+right)/2;
        if(valore < a[middle]) {
            return ricercaBinariaDic(len,a,valore,left,middle);
        }
        else {
            return ricercaBinariaDic(len,a,valore,middle,right);
        }
    }
}
```


Correttezza: caso base

Per ipotesi, l'array è ordinato (precondizione).

- Se $n==1$, allora ho $left+1==right$. Quindi il programma termina (non ci sono loop o chiamate) e ritorna $left$ se $valore==a[left]$, oppure -1 se $valore!=a[left]$

Correttezza: caso induttivo (strong induction)

Se $n > 1$ allora $left + 1 \neq right$, il primo `if` fallisce. Nel secondo `if` ho 2 possibilità:

1. `valore < a[middle]`: poiché l'array è ordinato, sappiamo che `valore` potrebbe essere solo in `a[left, middle)`. Ma, per ipotesi induttiva, la chiamata ricorsiva su `[left, middle)` dà il risultato corretto.
2. `valore >= a[middle]`: similmente, poiché l'array è ordinato, sappiamo che `valore` potrebbe essere solo in `a[middle, right)`. Ma, per ipotesi induttiva, la chiamata ricorsiva su `[middle, right)` dà il risultato corretto.